

Cours MERN Stack – Séance 2

Compte-rendu de TP

*Structuration professionnelle de l'API :
Architecture modulaire SoC*

Réalisé par : **Hedil Rouatbi**

Enseignant : Dr. Abdelweheb GUEDES

Octobre 2026

1 TP guidé : refactorisation de l'API Blog

1.1 Étapes 1 et 2 : dossiers de l'architecture et contrôleur

On crée les dossiers `controllers/` et `routes/`, puis on déplace les données et la logique métier des articles dans `controllers/articleController.js`.

```

1 // Données isolées dans le contrôleur (en mémoire pour cette séance)
2 let articles = [
3   { id: 1, title: 'Bienvenue sur le blog', author: 'Admin' },
4   { id: 2, title: 'Mon premier serveur Express', author: 'Aya' },
5   { id: 3, title: 'Tester une API avec Postman', author: 'Aya' }
6 ];
7
8 let prochainId = 4;
9
10 // 1. Récupérer tous les articles (avec filtre optionnel ?author=...)
11 const getAllArticles = (req, res) => {
12   const { author } = req.query;
13   let resultat = articles;
14
15   if (author) {
16     resultat = articles.filter(a => a.author === author);
17   }
18
19   res.status(200).json({
20     total: resultat.length,
21     articles: resultat
22   });
23 };
24
25 // 2. Récupérer un article par son ID unique
26 const getArticleById = (req, res) => {
27   const id = Number(req.params.id);
28   const article = articles.find(a => a.id === id);
29
30   if (!article) {
31     return res.status(404).json({
32       error: 'Article ${id} introuvable'
33     });
34   }
35
36   res.status(200).json(article);
37 };
38
39 // 3. Créer un nouvel article
40 const createArticle = (req, res) => {
41   const { title, author } = req.body;
42
43   if (!title || !author) {

```

Figure 1: Contrôleur des articles (`articleController.js`)

1.2 Étape 3 : le routeur des articles

Le routeur associe chaque verbe HTTP et chaque chemin relatif à la bonne fonction du contrôleur ; le préfixe `/api/articles` est défini dans `server.js`.

```

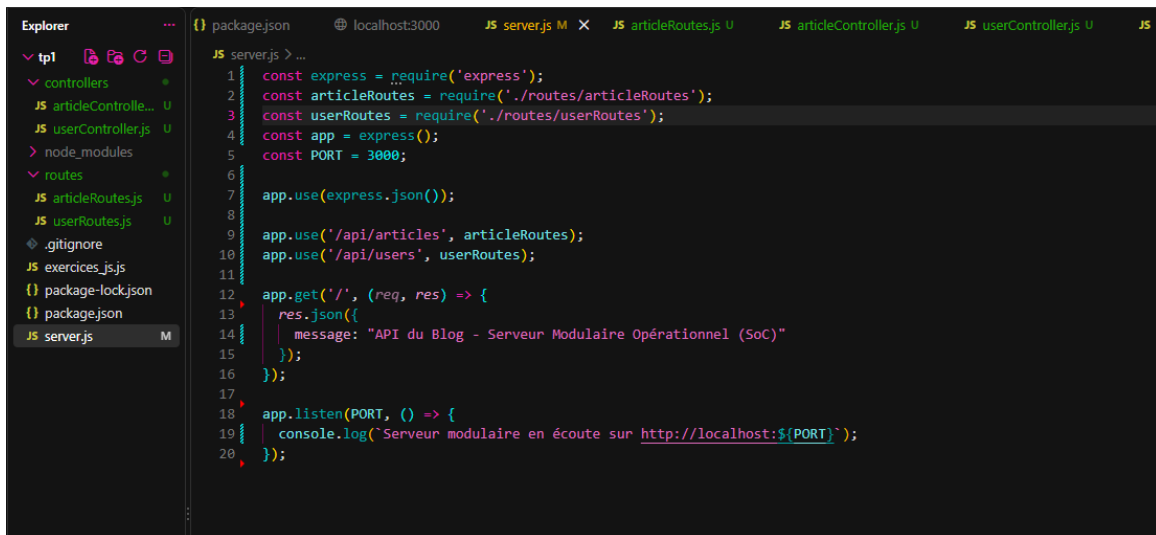
1 const express = require('express');
2 const router = express.Router();
3
4 const articleController = require('../controllers/articleController');
5
6 // Remarque capitale : Le préfixe '/api/articles' sera défini dans server.js.
7 // Ici, '/' représente donc la racine de la ressource : '/api/articles' !
8
9 router.get('/', articleController.getAllArticles);
10 router.get('/:id', articleController.getArticleById);
11 router.post('/', articleController.createArticle);
12 router.put('/:id', articleController.updateArticle);
13
14 module.exports = router;

```

Figure 2: Routeur des articles (`articleRoutes.js`)

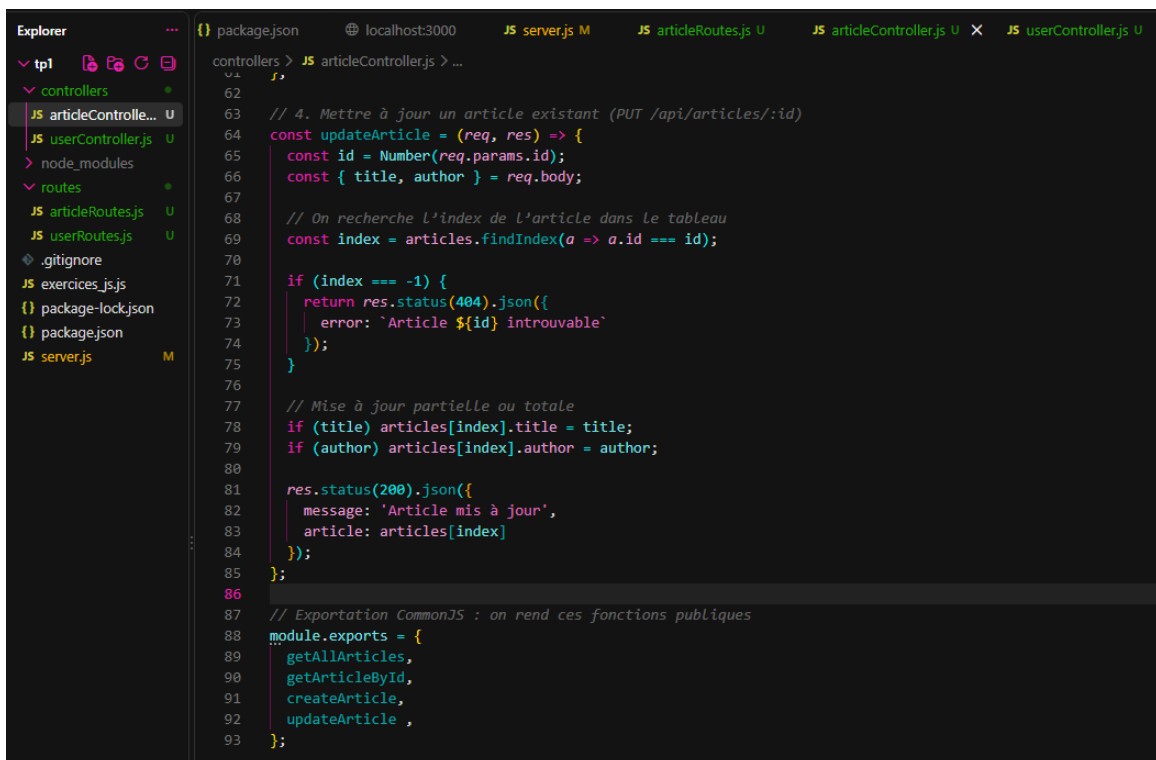
1.3 Étape 4 : allègement de server.js

`server.js` ne garde que la configuration globale : le middleware `express.json()`, le montage des routeurs et l'écoute du port.

Figure 3: Fichier `server.js` allégé

1.4 Étape 5 : modification avec PUT

La fonction `updateArticle` cherche l'article avec `findIndex`, renvoie 404 s'il n'existe pas, sinon met à jour le titre et/ou l'auteur.

Figure 4: Fonction `updateArticle` et export du contrôleur

2 Tests Postman : domaine articles

2.1 PUT /api/articles/2 : test réussi (200 OK)

On envoie un nouveau titre pour l'article 2 ; le serveur répond 200 avec l'article mis à jour.

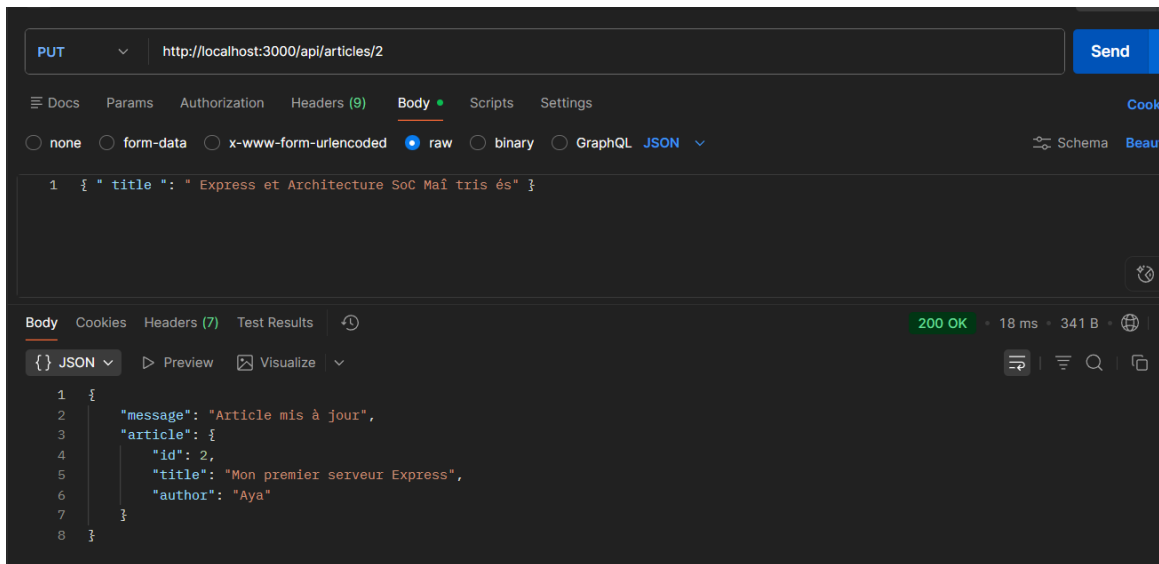


Figure 5: PUT réussi sur l'article 2

2.2 PUT /api/articles/99 : test d'échec (404 Not Found)

L'identifiant 99 n'existe pas : `findIndex` renvoie `-1` et le serveur répond 404 avec un message d'erreur.

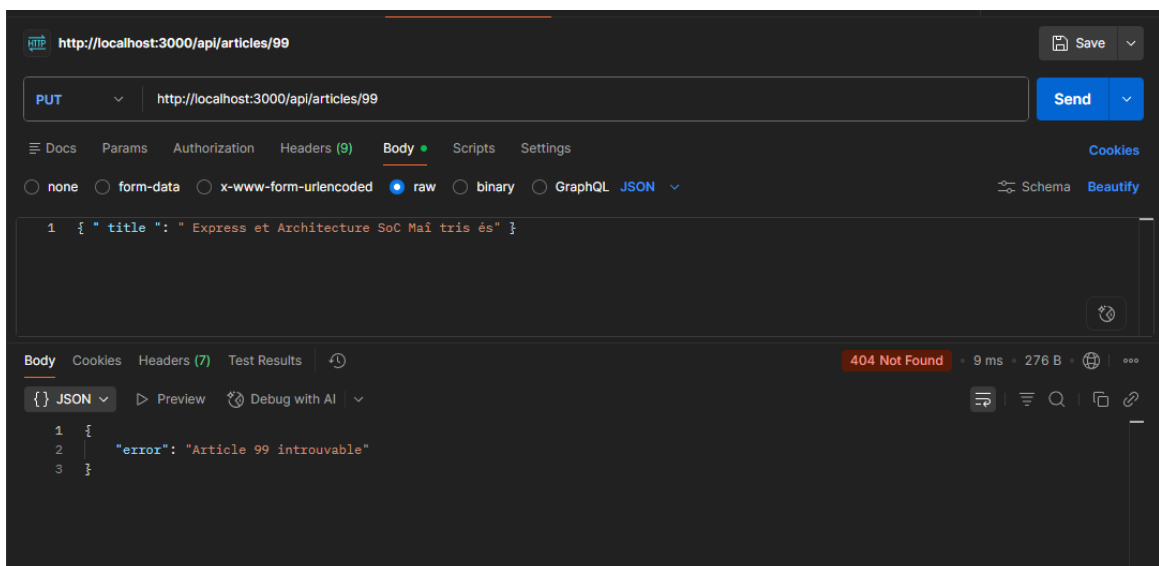


Figure 6: PUT en échec sur un identifiant inexistant

2.3 DELETE /api/articles/1 : test réussi (200 OK)

L'article 1 est supprimé grâce à `filter`, et le serveur confirme la suppression.

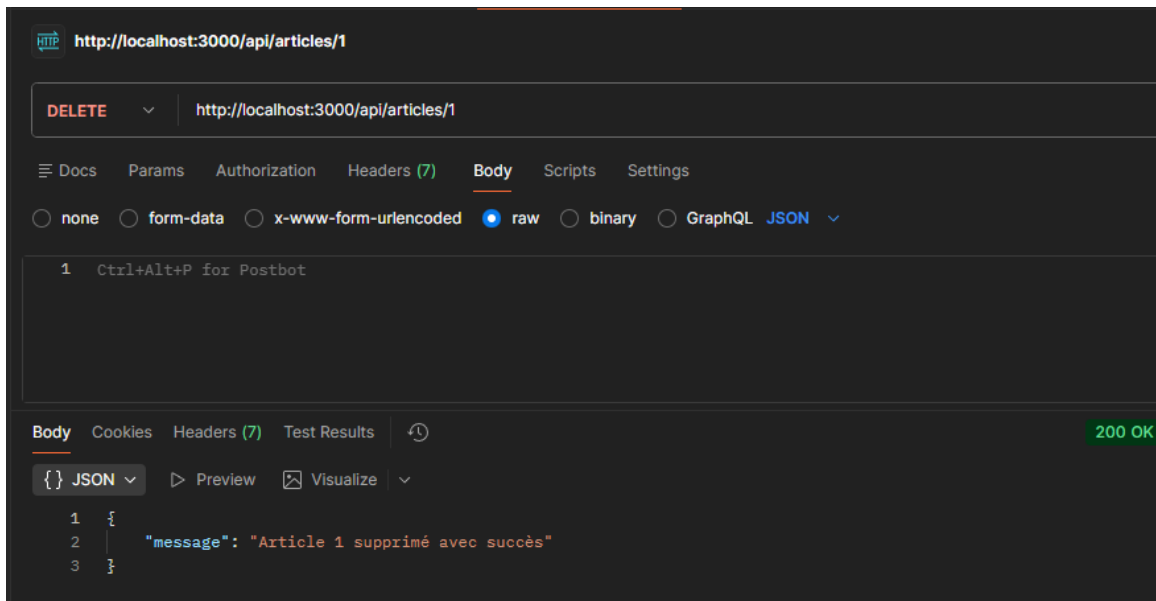


Figure 7: Suppression de l'article 1

3 Tests Postman : domaine users (exercice 1)

3.1 POST /api/users : création valide (201 Created)

Un nom et un email valides sont envoyés ; l'utilisateur est créé avec un nouvel identifiant.

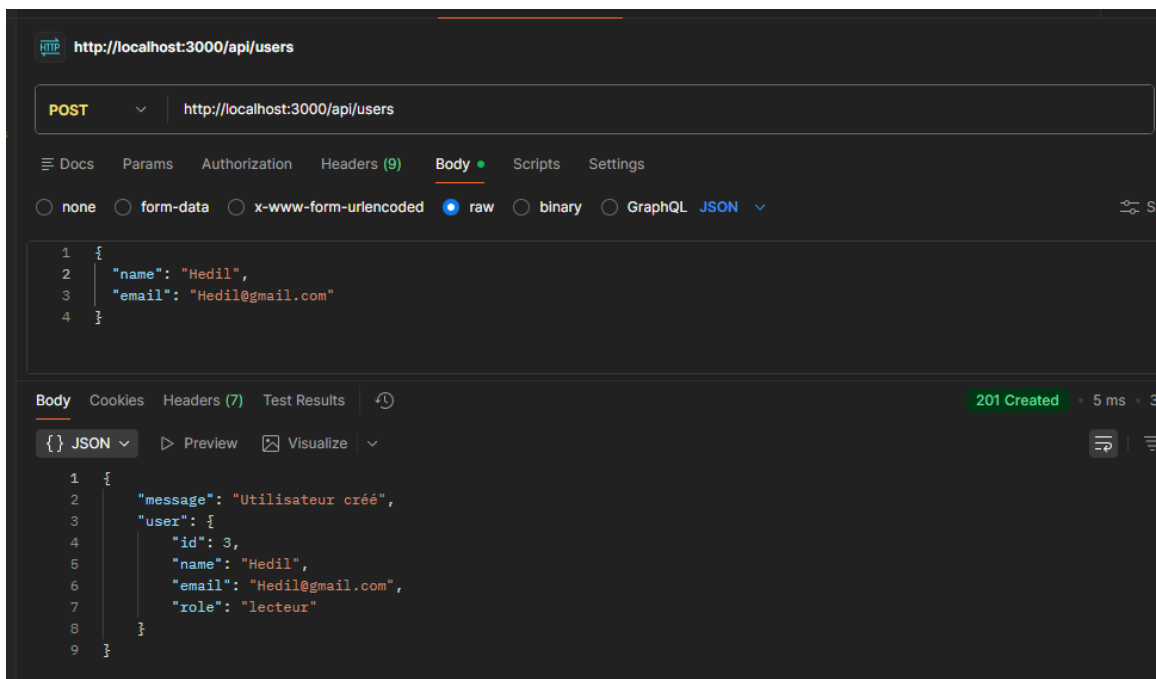


Figure 8: Création d'un utilisateur valide

4 Code source

Le code source complet du projet (`UserController.js`, `userRoutes.js`, `validators.js` et `server.js`) est disponible sur GitHub :

https://github.com/hedil1/TP1_MERN-Stack